École Nationale des Sciences Appliquées de Béni Mellal
Université Sultan Moulay Slimane

Filière : Intelligence Artificielle et Cybersécurité
Module : Blockchain

CrowdFund DAO

Plateforme de financement participatif décentralisée à gouvernance par jeton

Rapport de mini-projet

Réalisé par : Sabrine Ouarchane
[Nom du binôme]

Encadré par : [Nom de l'encadrant]

Dépôt Git : `[liendudÃľpÃŧt]`

Année universitaire 2026–2027

Table des matières

1 Introduction et cahier des charges **2**
1.1 Problème traité . . . 2
1.2 Principe . . . 2
1.3 Cahier des charges . . . 2
1.4 Choix de l'environnement : Hardhat . . . 2

2 Architecture **3**
2.1 Ordre de déploiement et rôles . . . 3

3 Choix de conception justifiés **3**

4 Description des contrats **4**
4.1 CFDToken – le jeton de gouvernance . . . 4
4.2 Crowdfunding – le cœur du financement . . . 5
4.3 Governance – la DAO . . . 5
4.4 MultiSigTreasury – le coffre 2-sur-3 . . . 6

5 Analyse de sécurité **6**
5.1 Reentrance . . . 6
5.2 Manipulation de la gouvernance . . . 7
5.3 Circuit breaker et contrôle d'accès . . . 7
5.4 Multisignature et anti-découpage . . . 7
5.5 Gestion des secrets . . . 7

6 Tests **8**
6.1 Fuzzing . . . 8

7 Audit statique (Slither) **9**

8 Interface dApp et stockage IPFS **10**
8.1 Écrans . . . 10
8.2 Cycle d'une transaction . . . 11
8.3 Métadonnées sur IPFS . . . 11

9 Déploiement sur le testnet Sepolia **12**

10 Bonus : détection d'anomalies (Python) **13**

11 Scénario de démonstration **14**

12 Limites, perspectives et conclusion **15**
12.1 Limites identifiées . . . 15
12.2 Perspectives . . . 15
12.3 Conclusion . . . 15

1 Introduction et cahier des charges

CrowdFund DAO est une application décentralisée (dApp) de financement participatif sans intermédiaire : les fonds sont gardés par des contrats intelligents, et les décisions sensibles sont votées par les contributeurs au lieu d'être prises par un administrateur. Le projet a été déployé et validé sur une blockchain Hardhat locale puis sur le testnet Sepolia.

1.1 Problème traité

Sur une plateforme centralisée, l'opérateur détient l'argent, décide seul des déblocages et peut être piraté, faire faillite ou censurer une campagne. Les contributeurs doivent lui faire confiance. Notre solution remplace cette confiance par des règles publiques et vérifiables, exécutées par la blockchain Ethereum.

1.2 Principe

Les porteurs de projet créent des campagnes. Les contributeurs financent en ETH et reçoivent en échange un jeton de gouvernance ERC-20, le **CFD**, proportionnel à leur apport. Deux décisions sont soumises au vote pondéré des détenteurs de CFD : le **déblocage de fonds** au bénéficiaire et le **gel** d'une campagne suspecte.

1.3 Cahier des charges

Module	**Exigence**	**Réalisation**
M1 – Jeton	ERC-20 « CFD » OpenZeppelin, frappe automatique à la contribution	`CFDToken` (ERC20 + ERC20Votes), 1 ETH = 1 000 CFD
M2 – Financement	`createCampaign`, `contribute`, 4 statuts, `refund`, `withdrawFunds` conditionné à la gouvernance	`Crowdfunding`
M3 – Gouvernance	Vote pondéré, quorum, majorité, délai avant exécution	`Governance` : quorum 20 %, majorité simple, timelock
M4 – dApp	Front-end + MetaMask + Ethers.js : campagnes, contribution, vote	React + Vite + Ethers.js v6, 3 onglets
Technique	Solidity ^0.8 + OpenZeppelin, Hardhat/Foundry, Sepolia, IPFS, fuzzing	Solidity 0.8.28, Hardhat 2, Pinata, 46 tests
Sécurité	Anti-reentrance, contrôle d'accès, circuit breaker, multisig 2-sur-3, snapshot, audit statique	Les 6 points (section 5), audit Slither (section 7)
Bonus	Détection d'anomalies en Python	3 règles + score de risque (section 10)

Table 1 – Cahier des charges et réalisation

1.4 Choix de l'environnement : Hardhat

Nous avons choisi **Hardhat** plutôt que Foundry pour trois raisons :

— les tests et le déploiement s'écrivent en JavaScript, comme le frontend, ce qui limite l'équipe à un seul langage applicatif ;

— Hardhat intègre Ethers.js, Chai et une blockchain locale avec manipulation du temps (`evm_increaseTime`), indispensable pour tester les dates limites et les votes ;

— sa documentation et sa communauté sont très fournies.

Foundry est plus rapide pour le fuzzing, mais un fuzzing à graine fixe en JavaScript a suffi à nos besoins (section 6.1).

2 Architecture

Le système répartit les responsabilités entre quatre contrats, pour qu'aucun ne cumule l'argent et le pouvoir de décision. Crowdfunding garde les fonds mais n'en libère aucun sans ordre de Governance ; Governance décide mais ne détient rien ; les gros montants passent en plus par le coffre multisig.

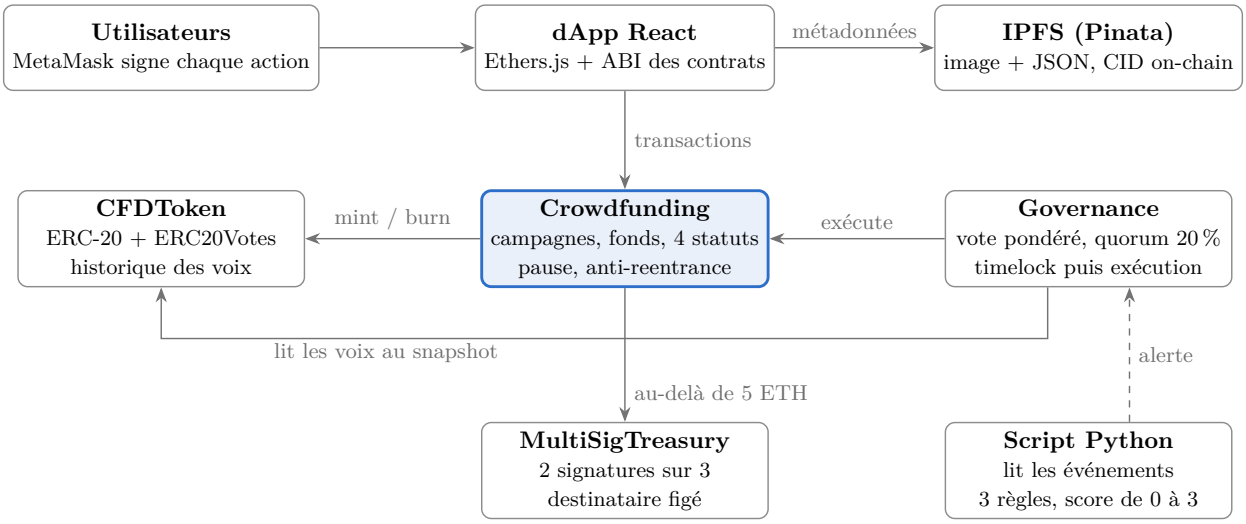


FIGURE 1 – Architecture : les fonds ne sortent de Crowdfunding qu'après un vote de la DAO

Trois briques restent hors chaîne : la dApp (aucun serveur, elle parle directement aux contrats via MetaMask), IPFS pour les métadonnées, et le script Python d'analyse, qui ne fait que lire l'historique et recommander un gel à la DAO (flèche en pointillés).

2.1 Ordre de déploiement et rôles

Chaque contrat doit connaître l'adresse de ceux dont il dépend, d'où un ordre imposé. Governance est déployé en dernier car il reçoit l'adresse de Crowdfunding ; Crowdfunding apprend ensuite qui est Governance par l'attribution d'un rôle, ce qui évite toute dépendance circulaire.

1. `CFDToken(admin)`
2. `MultiSigTreasury([3 gardiens], 2)`
3. `Crowdfunding(admin, token, treasury)`
4. `Governance(token, crowdfunding, dureeVote, delai)`
5. Attribution des rôles : `MINTER_ROLE` du jeton → Crowdfunding ; `setCore` du coffre → Crowdfunding ; `GOVERNANCE_ROLE` de Crowdfunding → Governance.
6. Renonciation du déployeur à `DEFAULT_ADMIN_ROLE` sur le jeton et sur Crowdfunding ; il ne conserve que `PAUSER_ROLE`.
7. Le script vérifie chaque rôle et écrit les adresses et les ABI dans `frontend/src/contracts/`.

Le dépôt suit la structure demandée : `/contracts` (4 contrats + contrat attaquant de test), `/test` (5 fichiers), `/frontend` (dApp React), `/scripts` (déploiement, avance du temps, simulation de fraude, détection Python) et `/docs` (rapports Slither et d'anomalies, adresses Sepolia).

3 Choix de conception justifiés

Chaque paramètre imposé « à définir et justifier » par le sujet a été fixé pour équilibrer sécurité et utilisabilité. Les durées de gouvernance sont des paramètres du constructeur, bornés dans le code entre 1 minute et 30 jours.

Paramètre	Valeur	Justification
Taux de frappe	1 ETH = 1 000 CFD	Proportionnel à l'apport. ETH et CFD ont 18 décimales : la conversion est une multiplication entière, sans arrondi. Le facteur 1 000 donne des soldes lisibles.
Quorum	20 % des CFD au snapshot	Empêche une petite minorité de décider seule sans exiger une participation irréaliste. Calculé avec `getPastTotalSupply` pour ne pas être modifiable après coup.
Majorité	OUI > NON (égalité = rejet)	Majorité simple des votes exprimés ; dans le doute, les fonds ne bougent pas.
Période de vote	3 jours (10 min sur Sepolia, 30 min en démo locale)	Laisse à chaque détenteur le temps de voter.
Timelock	1 jour (2 min en démo)	Délai entre adoption et exécution : permet de réagir à un vote malveillant, par exemple avec la pause.
Expiration	7 jours après le timelock	Empêche de ressortir un vieux vote, dans un contexte différent.
Seuil de proposition	100 CFD (0,1 ETH contribué)	Anti-spam. Le bénéficiaire peut toujours demander le déblocage de sa propre campagne.
Seuil multisig	5 ETH cumulés par campagne	Au-delà, les fonds passent par le coffre 2-sur-3. Comparé au cumul retiré + nouveau montant, pour empêcher le découpage.
Durée de campagne	1 heure à 90 jours	Évite les campagnes absurdes (durée nulle ou séculaire).

Table 2 – Paramètres de la plateforme

Burn au remboursement. Sans destruction des jetons, un contributeur pourrait récupérer ses ETH tout en gardant son pouvoir de vote. Le remboursement brûle donc exactement les CFD obtenus par cette contribution ; s'il les a transférés, le remboursement échoue, ce qui empêche de compter deux fois les mêmes voix.

Remboursement proportionnel. Si une campagne est gelée après un déblocage anticipé, il ne reste qu'une partie des fonds. Chaque contributeur récupère alors :

$$\text{remboursement} = \frac{\text{contribution} \times (\text{collecté} - \text{retiré})}{\text{collecté}}$$

Comme plus aucun dépôt ni retrait n'est possible dans cet état, ce pourcentage est identique pour tous, quel que soit l'ordre des remboursements. L'arrondi entier est toujours en défaveur de l'appelant : le contrat ne peut jamais rendre plus qu'il ne détient (propriété vérifiée par fuzzing).

Décentralisation. Après le déploiement, le déployeur renonce au rôle `DEFAULT_ADMIN_ROLE` sur le jeton et sur Crowdfunding. Il ne garde que `PAUSER_ROLE` : il peut suspendre la plateforme en cas d'incident, mais ne peut ni frapper de jetons, ni s'attribuer le rôle de gouvernance, ni déplacer des fonds.

4 Description des contrats

Le système compte quatre contrats Solidity 0.8.28 (compilés pour l'EVM Cancun, optimiseur activé à 200 runs), plus un contrat attaquant utilisé uniquement dans les tests.

4.1 CFDToken – le jeton de gouvernance

Hérite de `ERC20`, `ERC20Votes` et `AccessControl` d'OpenZeppelin v5.

— `mint(to, amount)` et `burn(from, amount)` sont protégées par `onlyRole(MINTER_ROLE)`, rôle détenu par le seul contrat Crowdfunding.

— `ERC20Votes` enregistre un *checkpoint* des soldes à chaque mouvement, ce qui fournit `getPastVotes(adresse, bloc)` et `getPastTotalSupply(bloc)` : c'est la base du snapshot.

— Avec ERC20Votes, un jeton non délégué ne compte pas comme voix. `mint` délègue donc automatiquement le contributeur à lui-même, avant la frappe, lors de sa première contribution.

— La surcharge de `_update` est obligatoire, car ERC20 et ERC20Votes la définissent tous deux.

4.2 Crowdfunding – le cœur du financement

Hérite de `AccessControl`, `ReentrancyGuard` et `Pausable`. Chaque campagne stocke : bénéficiaire, objectif, date limite, montant collecté, montant autorisé par la DAO, montant retiré, statut et CID IPFS. Les contributions sont mémorisées par campagne et par adresse.

Fonction	**Appelant**	**Effet**
`createCampaign(objectif, durée, bénéf., cid)`	tous	Crée une campagne En cours
`contribute(id)`	tous	Reçoit l'ETH, frappe 1 000 CFD par ETH
`finalize(id)`	tous, après la date limite	Réussie si l'objectif est atteint, sinon Échouée
`refund(id)`	contributeurs	Rembourse (Échouée ou Annulée), brûle les CFD
`approveRelease(id, montant)`	Governance uniquement	Autorise un retrait, cumulable, plafonné au collecté
`freezeCampaign(id)`	Governance uniquement	Passe la campagne à Annulée
`withdrawFunds(id, montant)`	bénéficiaire uniquement	Verse le montant autorisé, directement ou via le multisig
`pause()` / `unpause()`	`PAUSER_ROLE`	Circuit breaker

Table 3 – Fonctions du contrat Crowdfunding

Machine d'états. Une campagne naît *En cours*. À la date limite, `finalize` la fait passer à *Réussie* (objectif atteint) ou *Échouée*. Avant sa clôture, ou même une fois réussie, un vote de gel la fait passer à *Annulée*. Les retraits ne sont possibles qu'en état En cours ou Réussie ; les remboursements qu'en état Échouée ou Annulée. `finalize` est appelable par n'importe qui, car une blockchain n'exécute rien d'elle-même à une date donnée ; le résultat ne dépend pas de l'appelant.

4.3 Governance – la DAO

Aucun rôle d'administration : personne n'a de pouvoir particulier sur la gouvernance. Deux types de propositions : **Release** (déblocage d'un montant) et **Freeze** (gel).

1. `propose()` enregistre `snapshotBlock = block.number - 1` et la fin du vote.
2. `vote(id, oui/non)` : poids = `getPastVotes(votant, snapshotBlock)`, un seul vote par adresse.
3. Fin du vote : adoptée si la participation atteint le quorum et si OUI > NON, sinon *Rejetée*.
4. Timelock : la proposition est *En attente* et ne peut pas être exécutée.
5. Fenêtre de 7 jours : *Exécutable* par n'importe qui ; `execute()` appelle `approveRelease` ou `freezeCampaign`. Passé ce délai : *Expirée*.

L'état n'est pas stocké mais **calculé** par `state(id)` à partir de l'heure du bloc et des votes : il ne peut donc jamais être incohérent.

4.4 MultiSigTreasury – le coffre 2-sur-3

Trois gardiens, deux confirmations requises (valeurs `immutable`, contrôlées au constructeur : au moins 3 gardiens, pas de doublon, pas d'adresse nulle).

— `deposit(destinataire)` : appelée uniquement par Crowdfunding (adresse fixée une seule fois via `setCore`) ; crée une transaction en attente.

— `confirm`, `revoke`, `execute` : réservées aux gardiens ; `execute` exige au moins 2 confirmations.

— **Le destinataire est fixé par Crowdfunding** : les gardiens peuvent approuver ou bloquer un paiement, jamais le rediriger. Même deux gardiens complices ne peuvent pas détourner les fonds vers eux-mêmes. Contrepartie : si les gardiens refusent de signer, les fonds restent bloqués.

5 Analyse de sécurité

Les six exigences de sécurité du sujet sont implémentées et chacune est prouvée par au moins un test. Nous y ajoutons deux protections identifiées pendant la conception : l'anti-découpage des retraits et la renonciation aux rôles d'administration.

Exigence	**Notion**	**Implémentation**	**Preuve**
Anti-reentrance	Checks-Effects-Interactions + verrou	`nonReentrant` sur `contribute`, `refund`, `withdrawFunds`, `execute` ; soldes à zéro avant tout envoi	Test avec le contrat `ReentrancyAttacker`
Contrôle d'accès	Moindre privilège	`MINTER_ROLE`, `GOVERNANCE_ROLE`, `PAUSER_ROLE`, gardiens	Tests « un inconnu ne peut pas… »
Circuit breaker	Pausable	`whenNotPaused` sur création, contribution, remboursement, retrait	Test + démo dans la dApp
Multisignature	2-sur-3, destinataire figé	`MultiSigTreasury` au-delà de 5 ETH cumulés	10 tests + propriété 3 du fuzzing
Anti-manipulation	Snapshot des voix	`getPastVotes` au bloc de création −1 ; quorum via `getPastTotalSupply`	Test + démo « Mallory »
Audit statique	Analyse de code	Slither, 102 détecteurs	Chapitre 7
Anti-découpage	Contournement de seuil	Seuil comparé au cumul retiré + montant	Test dédié
Décentralisation	Point de défaillance unique	`renounceRole` sur jeton et cœur	Test + vérification dans `deploy.js`

Table 4 – Couverture des exigences de sécurité

5.1 Reentrance

L'attaque de reentrance (piratage de *The DAO*, 2016) exploite un contrat qui envoie de l'ETH avant de mettre à jour ses comptes : la fonction `receive()` de l'attaquant rappelle aussitôt la fonction de retrait, qui croit que rien n'a encore été versé. Dans `refund`, l'ordre est strictement *Checks*, *Effects*, puis *Interactions* :

```
function refund(uint256 id) external nonReentrant whenNotPaused campaignExists(id) {
    Campaign storage c = campaigns[id];
    // CHECKS
    require(c.status == Status.Failed || c.status == Status.Cancelled, "CF: remboursement impossible");
    uint256 contributed = contributions[id][msg.sender];
    require(contributed > 0, "CF: rien a rembourser");
    uint256 amount = (contributed * (c.raised - c.withdrawn)) / c.raised;
    // EFFECTS
    contributions[id][msg.sender] = 0;
    token.burn(msg.sender, contributed * RATE);
    // INTERACTIONS
    (bool ok, ) = payable(msg.sender).call{value: amount}("");
    require(ok, "CF: transfert echoue");
    emit Refunded(id, msg.sender, amount);
}
```

Le verrou `nonReentrant` ajoute une seconde barrière. Notre test déploie un contrat attaquant qui rappelle `refund` depuis `receive()` : la transaction entière est annulée, les 5 ETH du contrat restent en place et les contributeurs honnêtes sont toujours remboursés normalement. L'envoi utilise `call` plutôt que `transfer`, limité à 2 300 gas et susceptible de bloquer un destinataire qui est lui-même un contrat.

5.2 Manipulation de la gouvernance

Sans snapshot, une attaquante pourrait acquérir massivement des jetons juste après la création d'un vote (ou les emprunter : *flash loan*), voter, puis les revendre. Ici, le pouvoir de vote est lu au bloc précédant la proposition. Le choix de `block.number - 1` empêche d'obtenir des jetons et de proposer dans la même transaction. Un jeton transféré après le snapshot ne peut pas voter deux fois : son nouveau détenteur avait 0 voix à ce bloc. Dans notre test, Mallory contribue 20 ETH après la création de la proposition : elle détient bien 20 000 CFD, mais son vote est refusé (« aucune voix au snapshot »).

5.3 Circuit breaker et contrôle d'accès

Le rôle `PAUSER_ROLE` suspend la création, les contributions, les remboursements et les retraits. Il ne permet pas de déplacer des fonds : un administrateur compromis peut au pire geler temporairement la plateforme. Dans la dApp, le bouton « Pause d'urgence » n'est affiché qu'à l'admin, mais cette restriction est purement ergonomique : la véritable vérification est faite on-chain par `onlyRole`.

5.4 Multisignature et anti-découpage

Si la règle était « un retrait de plus de 5 ETH passe par le multisig », un bénéficiaire pourrait retirer 7 ETH en deux fois 3,5 ETH. Le contrat compare donc `retiré + montant` au seuil : dès que le cumul dépasse 5 ETH, le montant est déposé dans le coffre au lieu d'être versé au bénéficiaire. La signature d'un seul gardien ne suffit pas, un gardien ne peut pas signer deux fois, et une transaction exécutée ne peut pas être payée une seconde fois.

5.5 Gestion des secrets

La clé privée du compte de déploiement Sepolia et le jeton d'accès Pinata sont stockés dans des fichiers `.env` exclus du dépôt (vérifié avec `git check-ignore`). Une clé Pinata accidentellement exposée pendant le développement a été révoquée et remplacée. Les comptes Hardhat, dont les clés sont publiques, ne sont jamais utilisés sur Sepolia.

6 Tests

La suite compte **46 tests, tous verts** (`npx hardhat test`, environ 19 s), dont 3 propriétés de fuzzing vérifiées sur 75 scénarios aléatoires. Chaque test s'exécute sur une blockchain Hardhat jetable.

Fichier	Tests	Ce qui est vérifié
`CFDToken.test.js`	7	Nom et symbole ; mint par le minter ; refus du mint et du burn par un inconnu ; auto-délégation ; burn qui retire les voix ; snapshot
`MultiSigTreasury.test.js`	10	Refus de moins de 3 gardiens et des doublons ; `setCore` unique ; dépôt réservé à Crowdfunding ; 1 signature insuffisante ; paiement à 2/3 ; pas de double signature ; inconnu refusé ; révocation ; pas de double paiement
`Crowdfunding.test.js`	15	Création, durée invalide, contribution et CFD, date limite, finalisation, remboursement + burn, remboursement proportionnel, retrait sans vote refusé, contrôle d'accès, retrait direct, **anti-découpage**, **circuit breaker**, **attaque par reentrance**
`Governance.test.js`	11	Droit de proposer, anti-spam, **scénario complet**, **anti-manipulation**, double vote, vote hors délai, quorum, majorité, gel, expiration, **décentralisation**
`Fuzz.test.js`	3	Voir section 6.1

TABLE 5 – Répartition des 46 tests

Capture à insérer : `images/tests-46-passing.png`

FIGURE 2 – Exécution de la suite de tests : 46 tests réussis

6.1 Fuzzing

Le fuzzing génère des entrées aléatoires (montants au wei près, nombre et ordre des contributeurs, pourcentage débloqué) et vérifie qu'une propriété reste toujours vraie. Le générateur est à **graine fixe** (`FUZZ_SEED`, 2026 par défaut) : un échec est donc reproductible à l'identique. `loadFixture` restaure l'état déployé entre deux scénarios, ce qui rend l'exécution rapide (2 s pour 75 scénarios ; 600 scénarios avec `FUZZ_RUNS=200`).

1. **CFD = 1 000 × ETH.** Après 1 à 8 contributions aléatoires, chaque solde CFD et chaque pouvoir de vote vaut exactement 1 000 fois la somme contribuée ; le contrat détient exactement le total collecté.
2. **Un remboursement ne rend jamais plus que ce qui reste.** Après un déblocage aléatoire (0 à 99 %) puis un gel, et des remboursements dans un ordre aléatoire : chaque contributeur reçoit exactement sa part proportionnelle, jamais plus que sa mise ; tous les CFD sont brûlés ; le reste d'arrondi est inférieur à 1 wei par contributeur ; un second remboursement est refusé.
3. **Seuil multisig cumulé.** Pour une suite aléatoire de retraits, tout retrait qui fait dépasser 5 ETH cumulés arrive dans le coffre (et lui seul) ; on ne peut jamais retirer plus que le montant autorisé par la DAO.



FIGURE 3 – Exécution des trois propriétés de fuzzing

7 Audit statique (Slither)

Slither n'a relevé **aucun finding de gravité High ou Medium**. Sur les 15 findings initiaux, 3 ont été corrigés et les 12 restants sont justifiés ci-dessous. Commande utilisée (38 contrats analysés, 102 détecteurs) :

```
slither . -filter-paths "node_modules|contracts/test" -checklist
```

Le code d'OpenZeppelin (déjà audité) et le contrat attaquant des tests (volontairement vulnérable) sont exclus.

ID	Détecteur	Gravité	Décision	Justification
0	reentrancy-events (`Governance.execute`)	Low	Corrigé	L'événement était émis après l'appel externe. Aucun risque réel (appel vers notre propre contrat, `executed` déjà à vrai), mais l'émission a été déplacée dans la phase Effects pour respecter strictement CEI.
1 à 9	timestamp	Low	Justifié	`block.timestamp` sert aux dates limites et aux fenêtres de vote. Le producteur de bloc ne peut décaler l'heure que de quelques secondes, alors que nos durées minimales sont de 1 minute (vote) et 1 heure (campagne). Plusieurs lignes signalées ne comparent que des statuts : faux positifs.
10 à 12	low-level-calls	Info	Justifié	`call` est la méthode recommandée pour envoyer de l'ETH. Le booléen de retour est toujours vérifié et les fonctions sont protégées par CEI et `nonReentrant`.
13	naming-convention (`_core`)	Info	Corrigé	Paramètre renommé `newCore`.
14	unindexed-event-address	Info	Corrigé	Paramètre de `CoreSet` déclaré `indexed`.

TABLE 6 – Findings Slither et décisions

Après corrections, les 46 tests passent toujours et Slither ne signale plus que les 12 findings justifiés. Les rapports sont joints dans `/docs`.



FIGURE 4 – Résumé de l'analyse Slither après corrections (12 findings justifiés)

8 Interface dApp et stockage IPFS

Le frontend est une application React (Vite) sans serveur : elle parle directement aux contrats via **Ethers.js v6** et **MetaMask**, à partir des adresses et des ABI que le script de déploiement écrit dans `frontend/src/contracts/`.

8.1 Écrans

Zone	Fonctions	Exigence
Barre du haut	Connexion MetaMask, vérification et changement de réseau, adresse, soldes ETH et CFD, rôle détecté on-chain, bouton Pause d'urgence (admin)	M4, circuit breaker
Campagnes	Liste avec image, description, progression, statut, temps restant ; création ; boutons Contribuer, Finaliser, Me faire rembourser, Retirer, affichés selon les règles du contrat	M2, M4
Gouvernance	Création de proposition, état calculé par le contrat, barres OUI/NON, quorum, bloc du snapshot, poids de vote au snapshot, bouton Exécuter	M3, M4
Coffre multisig	Gardiens et règle 2/3, solde, transactions et signatures, boutons Signer, Retirer ma signature, Exécuter le paiement	Multisig

TABLE 7 – Écrans de la dApp

Lorsqu'un utilisateur n'avait aucun CFD au bloc du snapshot, l'onglet Gouvernance n'affiche pas les boutons de vote mais une explication : c'est ce que voit Mallory lors de la démonstration. La dApp suit en direct les changements de compte et de réseau dans MetaMask. Les erreurs des contrats (messages `require` et erreurs personnalisées d'OpenZeppelin, décodées grâce à l'ABI) sont traduites en français.



FIGURE 5 – Onglet Campagnes



FIGURE 6 – Onglet Gouvernance : votes, quorum et snapshot



FIGURE 7 – Onglet Coffre multisig : transaction à 2 signatures sur 2

8.2 Cycle d'une transaction

1. L'utilisateur clique, par exemple, sur « Contribuer 4 ETH ».
2. React appelle `crowdfunding.contribute(id, { value })` avec le *signer* MetaMask.
3. MetaMask affiche la transaction et ses frais ; l'utilisateur confirme.
4. La dApp attend la confirmation dans un bloc (`tx.wait()`).
5. Elle relit l'état des contrats (lectures gratuites, sans signature) et met à jour l'affichage.

8.3 Métadonnées sur IPFS

Stocker du texte et des images on-chain coûterait très cher. À la création d'une campagne, la dApp envoie d'abord l'image, puis un fichier JSON (`name`, `description`, `image`, `createdBy`, `createdAt`) sur IPFS via Pinata. **Seul le CID du JSON est stocké dans le contrat.** À l'affichage, la dApp lit le JSON et l'image via la passerelle Pinata. Si l'envoi IPFS échoue, aucune transaction n'est envoyée : une campagne n'existe jamais sans métadonnées.

Un CID est l'empreinte cryptographique du contenu : toute modification du fichier produirait un autre CID, qui ne correspondrait plus à celui enregistré on-chain. Les métadonnées sont donc infalsifiables sans payer leur stockage sur la blockchain.

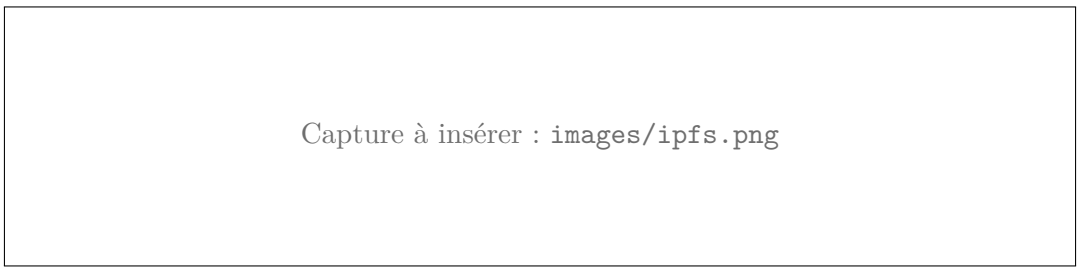


FIGURE 8 – Campagne avec image IPFS et fichier JSON ouvert sur la passerelle

Limite. Le jeton Pinata est nécessaire dans le navigateur pour envoyer des fichiers ; il est donc lisible par un utilisateur averti. Pour ce prototype, nous avons utilisé une clé dédiée et révocable ; en production, l'envoi passerait par un petit serveur intermédiaire qui garderait la clé secrète.

9 Déploiement sur le testnet Sepolia

Les quatre contrats ont été déployés sur Sepolia (chainId 11155111) le 5 novembre 2026 avec `npx hardhat run scripts/deploy.js -network sepolia`, pour environ 0,0155 ETH de test de frais. Le script vérifie ensuite chaque rôle : Crowdfunding détient `MINTER_ROLE`, Governance détient `GOVERNANCE_ROLE`, le coffre connaît Crowdfunding, le déployeur garde `PAUSER_ROLE` et n'a plus aucun rôle d'administration.

Contrat	**Adresse Sepolia**
CFDToken	`0x46161e71DDF8FcE90a105dc2bDfFA383A62Ce40b`
MultiSigTreasury	`0x0B72ECF150599ed91a75A4016f1e61fA1437e81d`
Crowdfunding	`0x3C5F137C139142Dd20Ce6c361D3685cdaCEF680f`
Governance	`0x03e67Af862702A04953860477AD152699CeC2409`

TABLE 8 – Adresses des contrats sur Sepolia (liens Etherscan)

Déployeur : `0x8b58666c252a24c8F326a0BC3e5d2Fc0f756589C`. Gardiens du multisig : trois comptes de test dédiés (`0x8b58...589C`, `0xE649...8f25`, `0x3412...920E`), jamais les comptes publics de Hardhat. Paramètres de démonstration : 10 minutes de vote et 2 minutes de timelock, car le temps ne peut pas être avancé sur un réseau public.

Scénario réel validé sur Sepolia. Création d'une campagne avec image et métadonnées IPFS ; contribution de 0,01 ETH (10 CFD frappés) ; proposition de déblocage par la bénéficiaire ; vote OUI ; attente réelle de la fin du vote et du timelock ; exécution ; retrait des fonds. Toutes les transactions sont consultables sur Etherscan.

Capture à insérer : `images/sepolia-deploiement.png`

FIGURE 9 – Sortie du script de déploiement sur Sepolia

Capture à insérer : `images/sepolia-etherscan.png`

FIGURE 10 – Transactions du contrat Crowdfunding sur Sepolia Etherscan

Capture à insérer : `images/sepolia-dapp.png`

FIGURE 11 – dApp connectée au réseau Sepolia

Le scénario multisig (retrait cumulé supérieur à 5 ETH) n'a pas pu être rejoué sur Sepolia faute d'ETH de test suffisant (les faucets limitent les dons à environ 0,05 ETH par jour). Il est démontré sur la blockchain locale et par les tests ; les trois gardiens configurés sont visibles dans le contrat déployé.

10 Bonus : détection d'anomalies (Python)

Le script `scripts/detect_anomalies.py` a classé la campagne frauduleuse simulée « suspecte » avec un score de 3/3, sans fausse alerte sur la campagne normale. Il lit l'historique des événements `Contributed` du contrat avec **web3.py**, applique trois règles par campagne et écrit un rapport JSON (`docs/anomalies-report.json`).

Règle	**Déclencheur**	**Fraude visée**
R1 – Montants répétitifs	≥ 3 adresses distinctes envoient exactement le même montant (au wei près)	Faux comptes pilotés par un même script
R2 – Adresses liées	≥ 3 contributeurs ont reçu leur premier ETH de la même adresse	Une seule personne derrière plusieurs comptes
R3 – Pic d'activité	≥ 5 contributions en moins de 10 minutes (fenêtre glissante)	Popularité gonflée artificiellement

TABLE 9 – Règles de détection

Le **score de risque** est le nombre de règles déclenchées : 0 = normale, 1 = à surveiller, 2 ou plus = suspecte, avec la recommandation de soumettre un gel à la DAO.

Validation. Le script `scripts/simuler-fraude.js` crée sur la blockchain locale une campagne normale (5 contributions de montants variés, espacées de 2 heures) et une campagne frauduleuse : Mallory, bénéficiaire de sa propre campagne, finance 6 comptes neufs qui lui envoient chacun 0,5 ETH en quelques secondes.

Campagne	**Contributions**	**Total**	**Règles**	**Score**	**Verdict**
#0 normale	5	5,5 ETH	aucune	0/3	normale
#1 suspecte	6	3 ETH	R1, R2, R3	3/3	suspecte, gel recommandé

TABLE 10 – Résultat de la détection sur les données simulées

La règle R2 révèle que l'adresse qui a financé les 6 contributeurs (`0x2361...1E8f`) est la bénéficiaire de la campagne elle-même : il s'agit d'un auto-financement déguisé. Il vise à faire paraître la campagne populaire, mais aussi à accumuler du pouvoir de vote : les 6 faux comptes

détiennent 3 000 CFD, soit 35 % des voix de la plateforme. La détection protège donc l'argent des contributeurs **et** la gouvernance.



FIGURE 12 – Sortie du script de détection d'anomalies

Boucle complète. Le script ne gèle rien lui-même : il recommande, et ce sont les détenteurs de CFD qui décident. Dans la démonstration, une proposition « Geler la campagne #1 » est votée puis exécutée : la campagne passe à *Annulée* et Mallory ne peut plus retirer les fonds.

Limites. Ces règles peuvent produire des faux positifs (plusieurs vrais donateurs choisissant tous « 1 ETH »), et un fraudeur prévenu peut varier ses montants. La règle R2 parcourt les blocs un à un : elle est désactivée automatiquement au-delà de 20 000 blocs, donc sur Sepolia sans indexeur. Les seuils sont des paramètres réglables.

11 Scénario de démonstration

La démonstration rejoue, sur la blockchain locale, un scénario qui exerce toutes les exigences du sujet en une dizaine de minutes. Amina porte un projet de panneaux solaires pour une coopérative ; Youssef, Salma et Karim la financent ; Mallory tente de manipuler la plateforme. Le script `avancer-temps.js` avance l'horloge de la blockchain pour franchir dates limites, votes et timelock.

1. **Déploiement** des 4 contrats, distribution des rôles, renonciation de l'admin à ses super-rôles.
2. **Création** : Amina crée la campagne (objectif 10 ETH) ; image et description partent sur IPFS, seul le CID est enregistré.
3. **Contributions** : Youssef 4 ETH, Salma 3 ETH, Karim 2 ETH, soit 4 000, 3 000 et 2 000 CFD.
4. **Déblocage anticipé** : Amina propose de débloquer 3 ETH. Le snapshot est pris au bloc précédent.
5. **Attaque éclair** : Mallory contribue 20 ETH pour obtenir 20 000 CFD et renverser le vote. La dApp affiche qu'elle n'avait aucune voix au snapshot : l'attaque échoue.
6. **Vote** : 7 000 OUI contre 2 000 NON, quorum atteint. Après le vote et le timelock, n'importe qui exécute ; Amina retire 3 ETH directement (sous le seuil).
7. **Retrait multisig** : la DAO autorise le solde ; le cumul dépassant 5 ETH, le retrait part dans le coffre. Un gardien signe (1/2, paiement impossible), un second signe (2/2), le paiement est exécuté.
8. **Campagne échouée** : une campagne n'atteint pas son objectif ; après finalisation, chaque contributeur est remboursé et ses CFD sont brûlés.
9. **Circuit breaker** : l'admin active la pause ; contributions et retraits sont refusés, puis rétablis.
10. **Fraude détectée** : le script Python signale la campagne de Mallory (score 3/3) ; la DAO vote son gel ; la campagne passe à Annulée.

L'attaque par reentrance n'est pas rejouée dans l'interface (elle nécessite un contrat attaquant) : elle est montrée par son test, qui échoue comme prévu.

12 Limites, perspectives et conclusion

12.1 Limites identifiées

— **Gros détenteurs (« baleines »).** Le snapshot bloque l'acquisition éclair de jetons, pas l'accumulation durable. Un contributeur qui détient la majorité des CFD avant un vote peut le décider seul. Le quorum, le timelock et la pause limitent l'impact sans l'empêcher.

— **Vote global.** Tous les détenteurs de CFD votent sur toutes les campagnes, même celles qu'ils n'ont pas financées.

— **Disponibilité du multisig.** Si deux gardiens refusent de signer ou perdent leurs clés, les fonds déposés dans le coffre restent bloqués.

— **Horodatage.** Les délais reposent sur `block.timestamp`, légèrement manipulable par le producteur de bloc (quelques secondes), négligeable face à nos durées.

— **Clé IPFS côté navigateur** et **règles de détection simples** (faux positifs possibles), détaillées aux sections précédentes.

— **Contrats non upgradables.** Une faille découverte après déploiement ne peut pas être corrigée sur place : il faut mettre en pause, puis redéployer et migrer.

12.2 Perspectives

— Vote quadratique ou plafonnement du poids par adresse, pour réduire l'influence des baleines.

— Utilisation du `Governor` et du `TimelockController` d'OpenZeppelin, standards audités.

— Déblocage par jalons (*milestones*) votables, et rotation des gardiens du multisig par vote.

— Vérification des contrats sur Etherscan et audit complémentaire avec Mythril ou un fuzzing Foundry.

— Détection d'anomalies par apprentissage (par exemple Isolation Forest) une fois un historique réel disponible, et indexation des événements (The Graph) pour l'appliquer sur le réseau public.

12.3 Conclusion

CrowdFund DAO répond à l'ensemble du cahier des charges : un jeton ERC-20 de gouvernance, un cœur de financement à quatre statuts, une DAO avec quorum, majorité et timelock, une dApp complète connectée à MetaMask et des métadonnées sur IPFS. Les six exigences de sécurité sont implémentées et prouvées par 46 tests, dont des attaques simulées et trois propriétés de fuzzing ; l'audit Slither ne relève aucun finding critique. Le système fonctionne sur le testnet Sepolia, et le bonus Python détecte un auto-financement déguisé que la DAO peut ensuite geler.

Au-delà du code, ce projet nous a montré que la sécurité d'une application décentralisée se joue autant dans la conception (ordre des opérations, seuils cumulés, snapshot, renonciation aux privilèges) que dans l'implémentation : plusieurs failles ont été évitées parce qu'elles ont été identifiées avant d'écrire la moindre ligne.